



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

**Институт информационных технологий (ИИТ)
Кафедра цифровой трансформации (ЦТ)**

ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ
по дисциплине «Разработка баз данных»

Практическое занятие № 6

Студенты группы *ИКБО-42-23 Голев С.С.*

(подпись)

Ассистент *Морозов Д.В.*

(подпись)

Отчет представлен «__» _____ 2025 г.

Москва 2025 г.

СОДЕРЖАНИЕ

ЗАДАНИЕ.....	3
ВЫПОЛНЕНИЕ ЗАДАНИЯ.....	5
ТРИГГЕРЫ.....	5
Сложная валидация отзыва.....	5
Поддержание согласованности.....	6
Аудит изменения статуса заказа.....	8
Защита данных.....	9
КУРСОРЫ.....	10

ЗАДАНИЕ

Задание №1: триггеры

Проанализировать предметную область своей базы данных и выявить не менее трёх бизнес-правил, реализация которых в виде ограничений целостности возможна только с помощью триггеров.

ВАЖНО: триггеры должны быть разными.

Для каждого правила:

- описать алгоритм его работы, указав таблицу, событие и последовательность действий;
- написать код триггерной функции на PL/pgSQL и оператор CREATE TRIGGER;
- продемонстрировать работу триггера на примерах DML-операций, которые как успешно выполняются, так и корректно прерываются триггером (два запроса).

Как выбрать сценарии для триггеров?

Чтобы вам было проще определить бизнес-правила для вашей базы данных, ищите сценарии, которые подпадают под следующие стандартные категории:

1. Аудит и логирование.

Запись факта изменения данных. Пример: при любом изменении столбца salary в таблице employees создавать запись в salary_log, сохраняя OLD.salary, NEW.salary и CURRENT_USER.

2. Сложная валидация (проверка).

Запрет операции на основе данных из другой таблицы.

Пример: триггер BEFORE INSERT на таблицу order_items должен проверить, что NEW.quantity не превышает quantity_on_hand в таблице products. Если превышает – вызвать RAISE EXCEPTION.

3. Поддержание согласованности (денормализация).

Автоматическое обновление связанных данных.

Пример: триггер AFTER INSERT на таблицу sales автоматически обновляет столбец total_spent в таблице customers.

4. Защита данных.

Запрет определенных операций.

Пример: триггер BEFORE DELETE на таблицу departments может запретить удаление, если в таблице employees еще есть сотрудники, ссылающиеся на этот отдел.

Задание №2: курсоры

Разработать два скрипта на PL/pgSQL, демонстрирующих оба способа обработки данных:

- С использованием явного курсора (DECLARE/OPEN/FETCH/CLOSE).
- С использованием неявного курсора (цикл FOR...IN).

Каждый SQL-запрос сопровождать комментарием, объясняющим его назначение и логику работы с учетом специфики вашей базы данных.

ВЫПОЛНЕНИЕ ЗАДАНИЯ

ТРИГГЕРЫ

Сложная валидация отзыва

```
CREATE OR REPLACE FUNCTION trg_review_validate_purchase()
RETURNS TRIGGER AS $$
DECLARE
    v_cnt INT;
BEGIN
    IF NEW.rating < 1 OR NEW.rating > 5 THEN
        RAISE EXCEPTION
            'Рейтинг должен быть в диапазоне от 1 до 5. Передано: %',
            NEW.rating;
    END IF;

    SELECT COUNT(*) INTO v_cnt
    FROM ordering o
    JOIN rtl_ordering ro
        ON ro.id_ordering = o.id_ordering
    WHERE o.id_client = NEW.id_client
        AND ro.id_rtl = NEW.id_rtl
        AND o.status <> 'cancelled';

    IF v_cnt = 0 THEN
        RAISE EXCEPTION
            'Клиент % не может оставить отзыв на товар %, т.к. не покупал его',
            NEW.id_client, NEW.id_rtl;
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

Рисунок 1 – триггерная функция

```
CREATE OR REPLACE TRIGGER trg_review_before_insert_validate
BEFORE INSERT ON review
FOR EACH ROW
EXECUTE FUNCTION trg_review_validate_purchase();
```

Рисунок 2 – привязка триггера

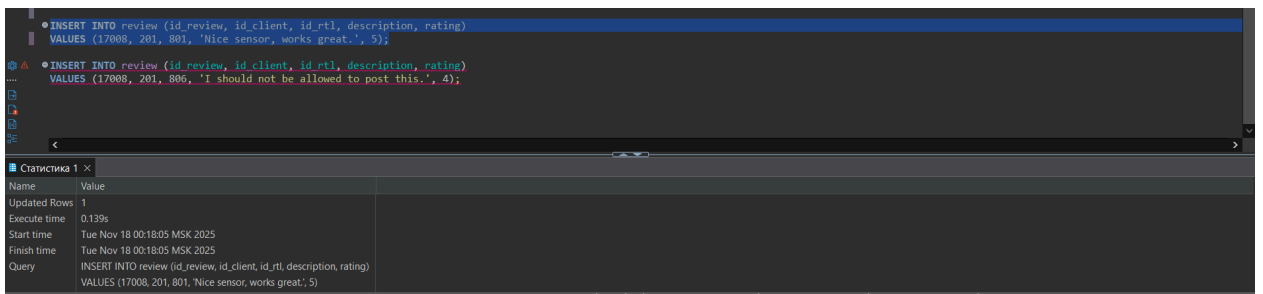


Рисунок 3 – тестирование триггера

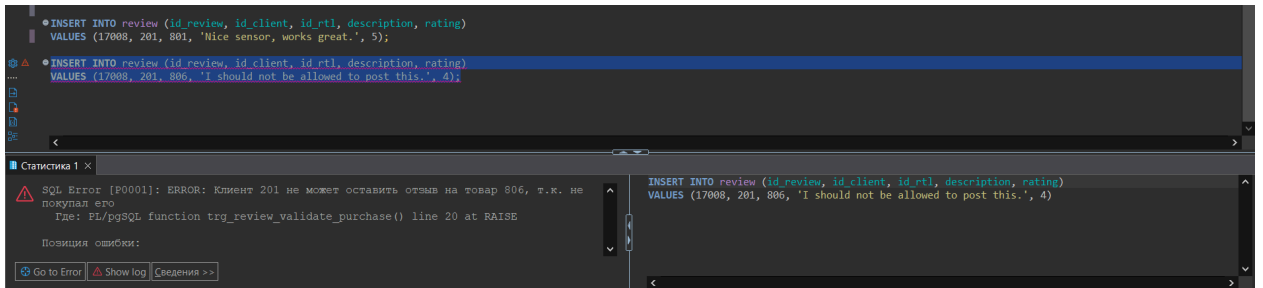


Рисунок 4 – тестирование триггера

Поддержание согласованности

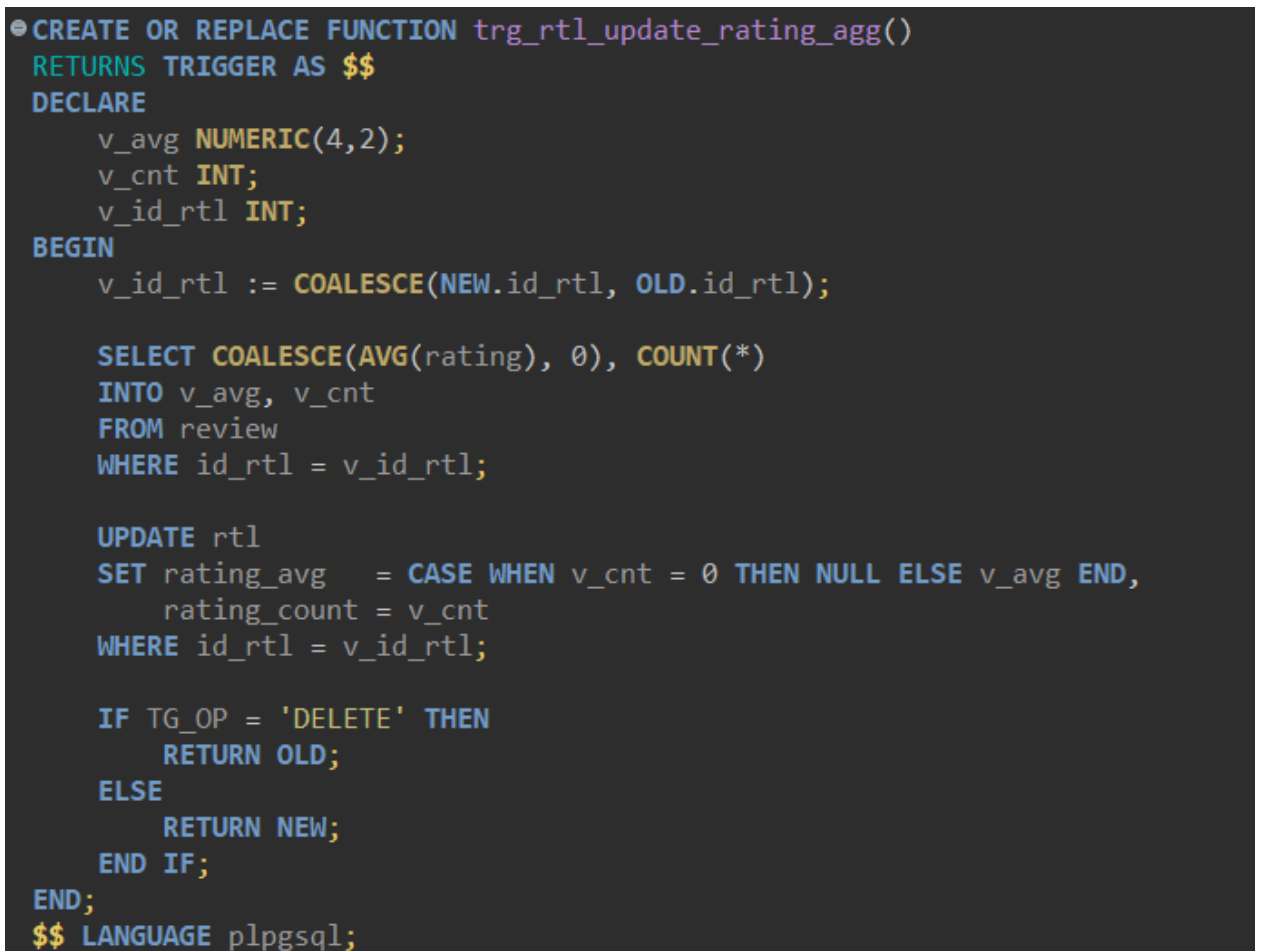
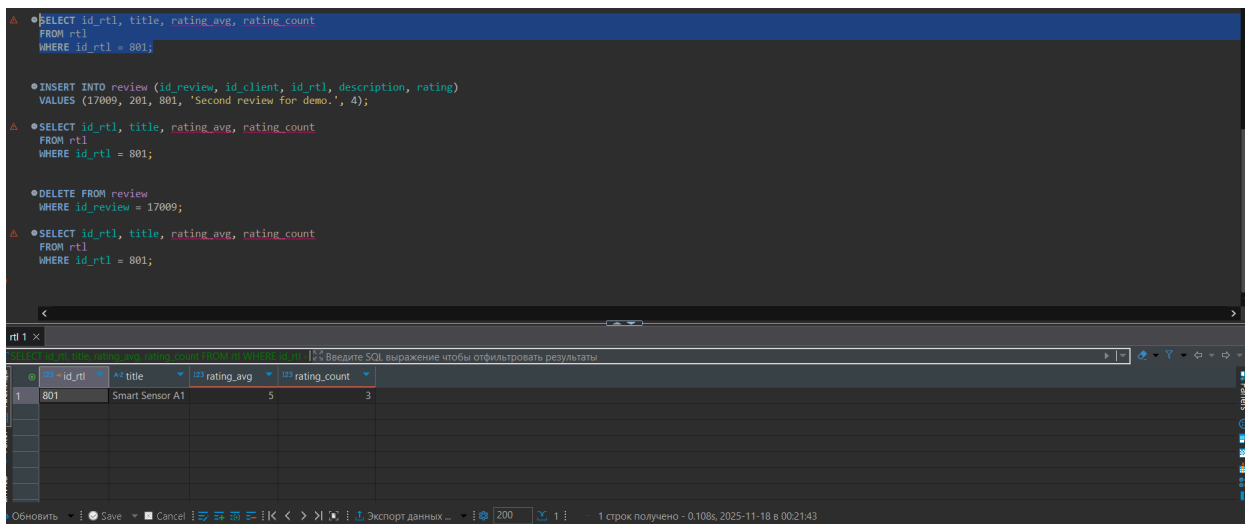


Рисунок 5 – код триггера

```
CREATE OR REPLACE TRIGGER trg_rtl_after_review_change
AFTER INSERT OR UPDATE OR DELETE ON review
FOR EACH ROW
EXECUTE FUNCTION trg_rtl_update_rating_agg();
```

Рисунок 6 – привязка триггера



```
SELECT id_rtl, title, rating_avg, rating_count
FROM rtl
WHERE id_rtl = 801;

INSERT INTO review (id_review, id_client, id_rtl, description, rating)
VALUES (17009, 201, 801, 'Second review for demo.', 4);

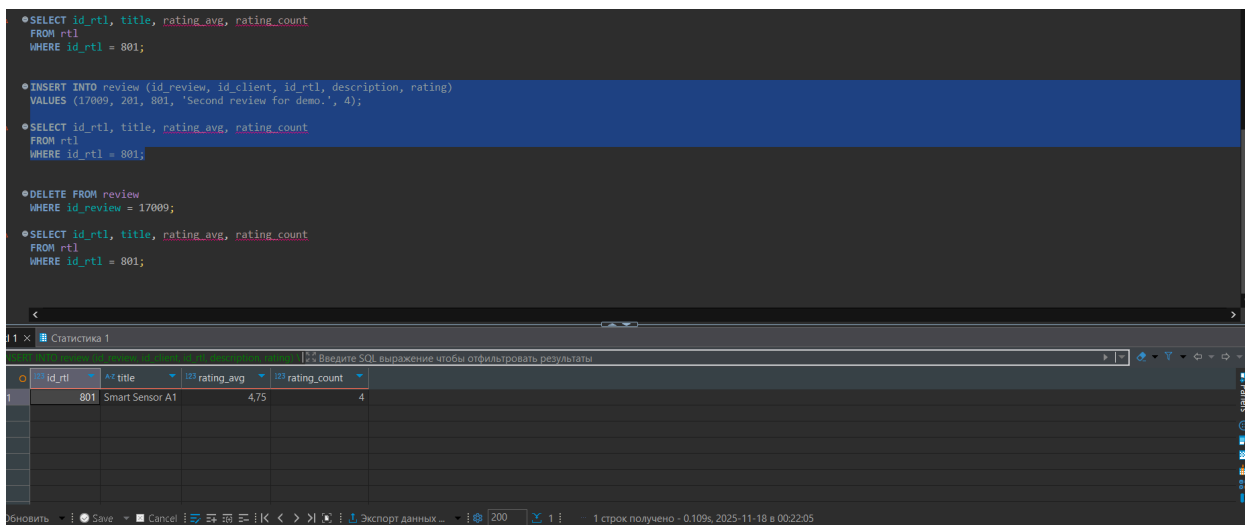
SELECT id_rtl, title, rating_avg, rating_count
FROM rtl
WHERE id_rtl = 801;

DELETE FROM review
WHERE id_review = 17009;

SELECT id_rtl, title, rating_avg, rating_count
FROM rtl
WHERE id_rtl = 801;
```

	id_rtl	title	rating_avg	rating_count
1	801	Smart Sensor A1	5	3

Рисунок 7 – демонстрация работы



```
SELECT id_rtl, title, rating_avg, rating_count
FROM rtl
WHERE id_rtl = 801;

INSERT INTO review (id_review, id_client, id_rtl, description, rating)
VALUES (17009, 201, 801, 'Second review for demo.', 4);

SELECT id_rtl, title, rating_avg, rating_count
FROM rtl
WHERE id_rtl = 801;

DELETE FROM review
WHERE id_review = 17009;

SELECT id_rtl, title, rating_avg, rating_count
FROM rtl
WHERE id_rtl = 801;
```

	id_rtl	title	rating_avg	rating_count
1	801	Smart Sensor A1	4.75	4

Рисунок 8 – демонстрация работы

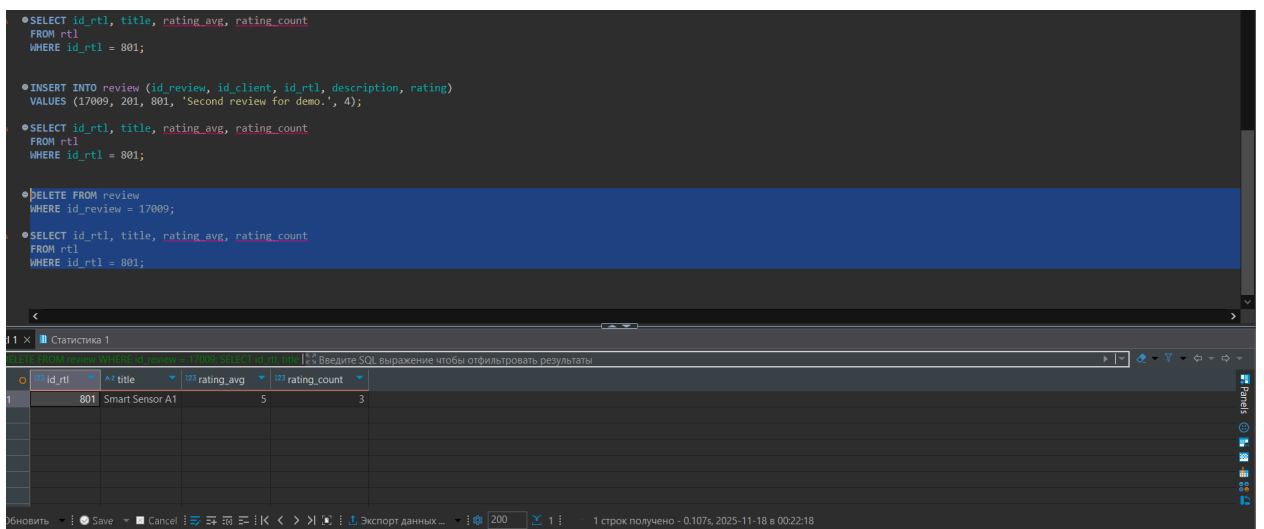


Рисунок 9 – демонстрация работы

Аудит изменения статуса заказа

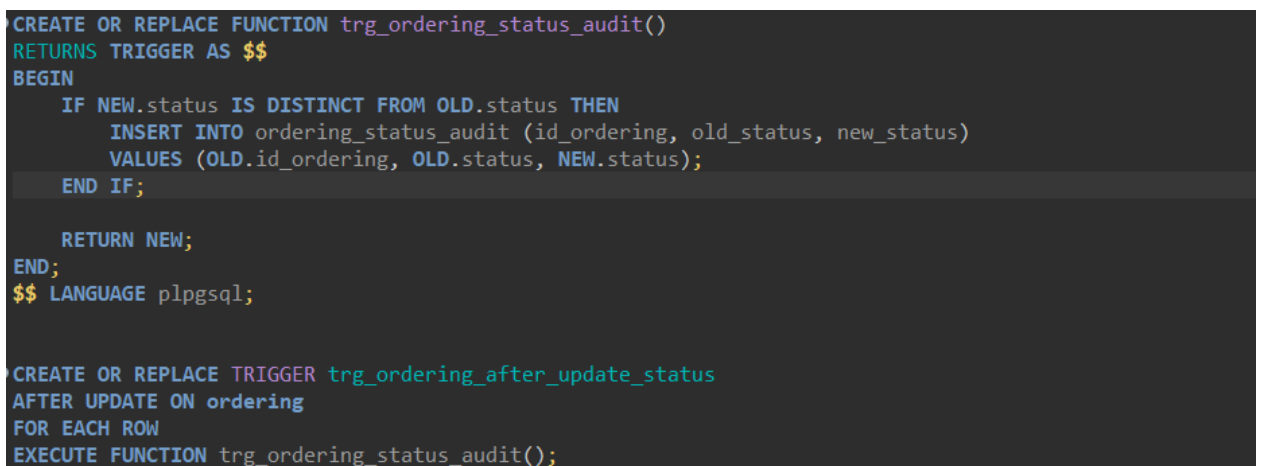


Рисунок 10 – триггерная функция и привязка триггера

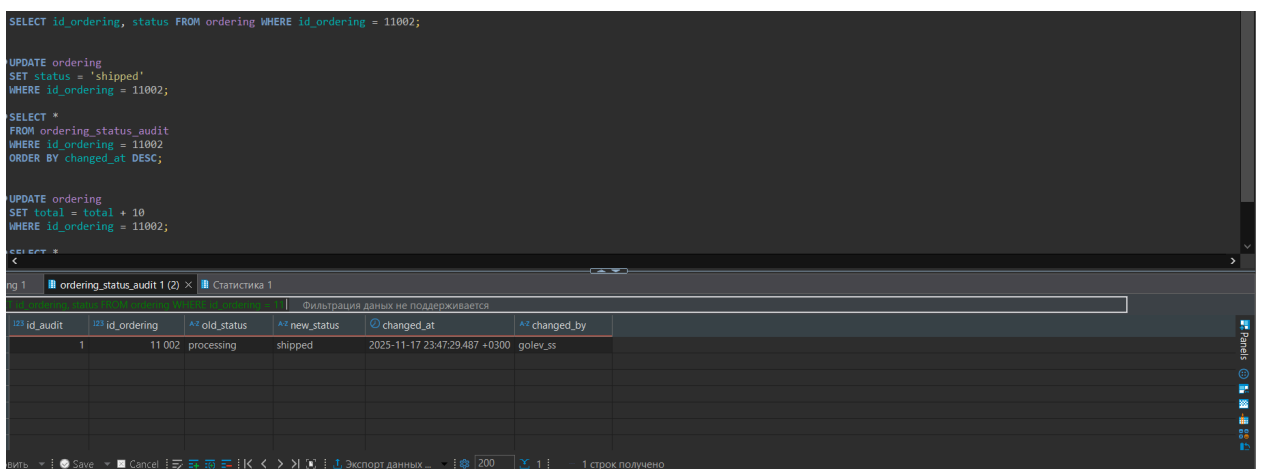


Рисунок 11 – Демонстрация работы

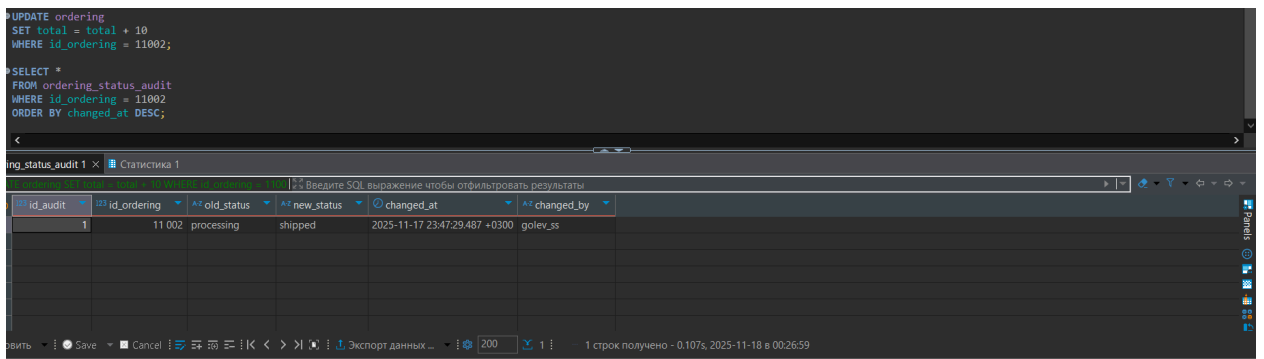


Рисунок 12 – Демонстрация работы

Защита данных

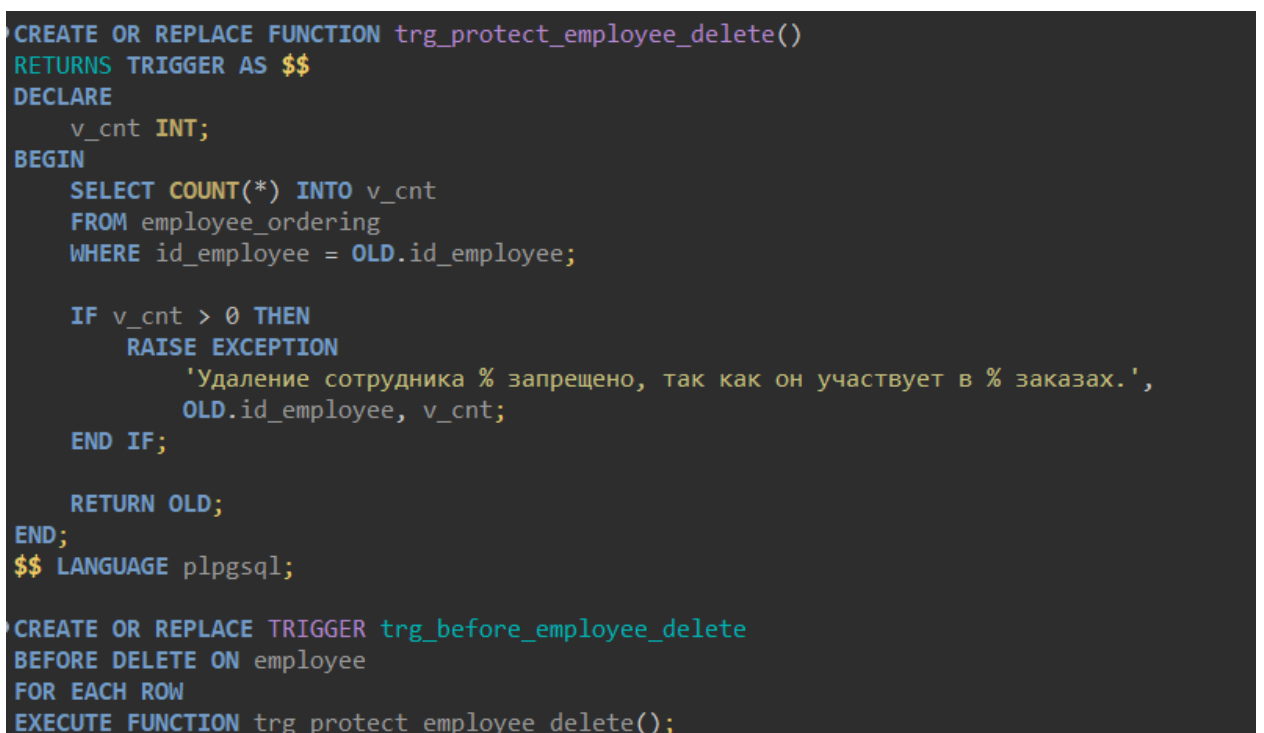


Рисунок 13 – Триггер функция и привязка

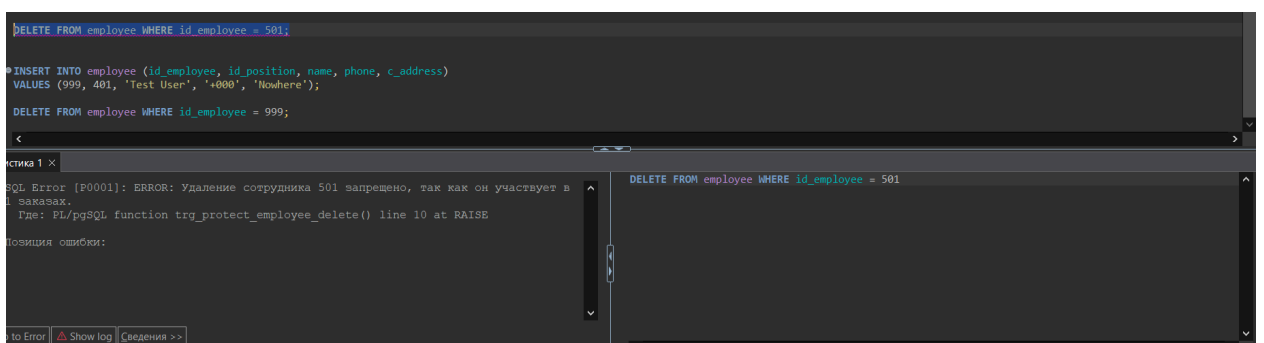


Рисунок 14 – Демонстрация работы

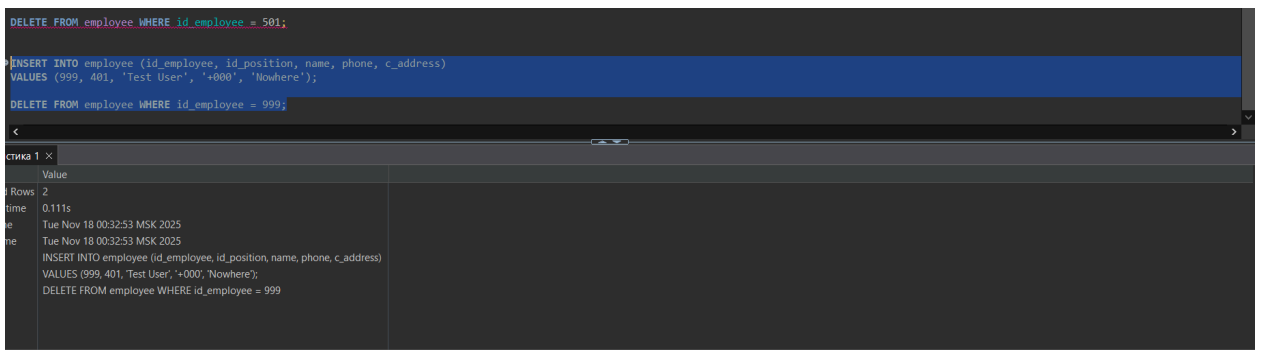


Рисунок 15 – Демонстрация работы

КУРСОРЫ

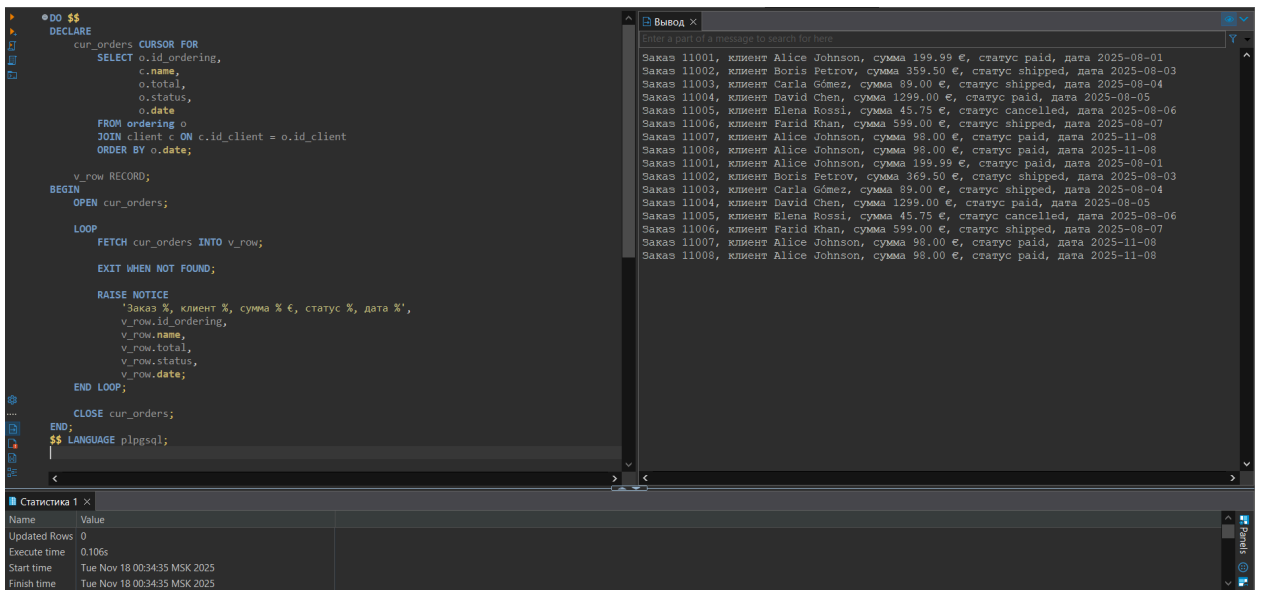


Рисунок 16 – Код курсора и демонстрация работы

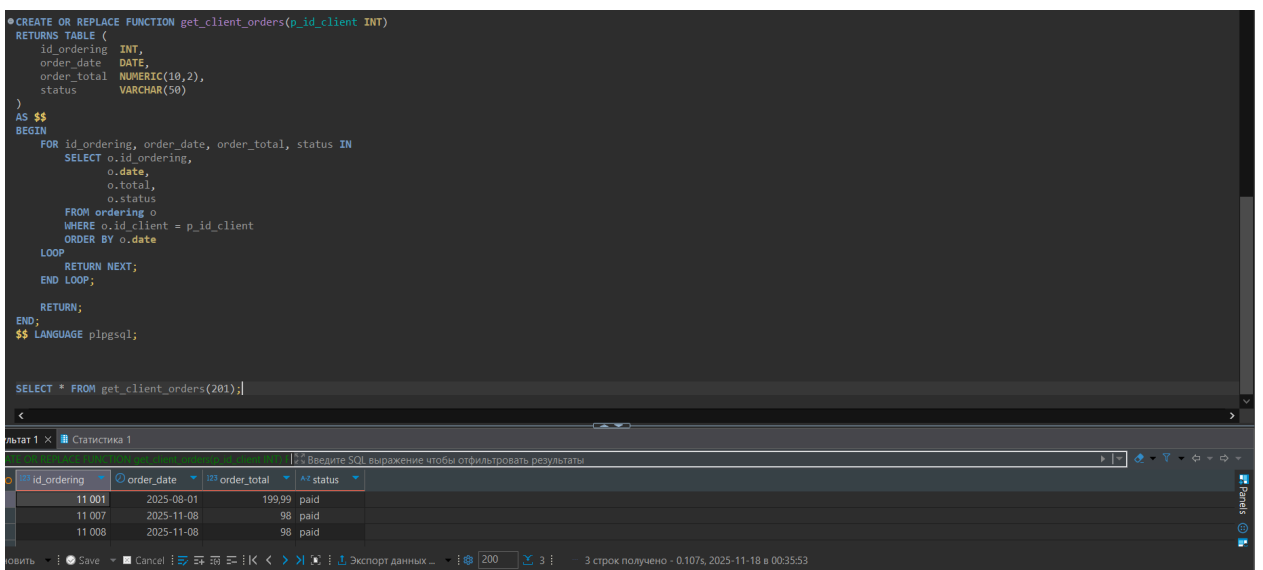


Рисунок 17 – Код неявного курсора